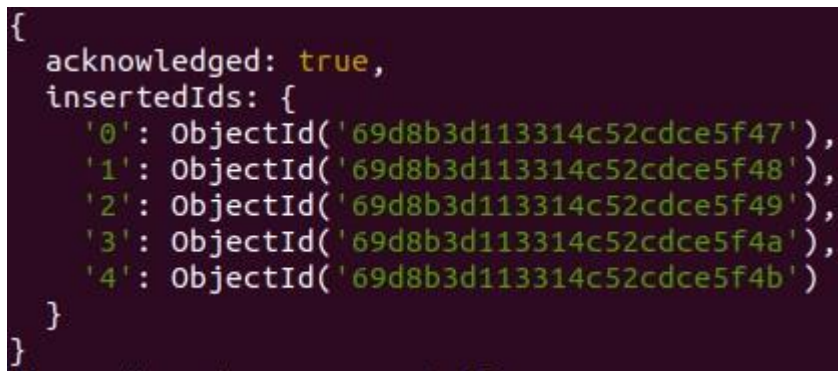


Practical No. 10

- `ngi@ngi-HP-EliteDesk-800-G2-SFF:~$ mongosh`
- `test> use dlog`
switched to db dlog

1. Sample Dataset (Insert Data):

```
db.orders.insertMany(
([
{ customer: "Amit", city: "Pune", amount: 500, status: "shipped", date: new Date("2025-01-10") },
{ customer: "Riya", city: "Mumbai", amount: 1200, status: "pending", date: new Date("2025-01-12") },
{ customer: "Amit", city: "Pune", amount: 700, status: "shipped", date: new Date("2025-02-01") },
{ customer: "John", city: "Delhi", amount: 300, status: "cancelled", date: new Date("2025-02-05") },
{ customer: "Riya", city: "Mumbai", amount: 900, status: "shipped", date: new Date("2025-02-10") }
])
```



```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d8b3d113314c52cdce5f47'),
    '1': ObjectId('69d8b3d113314c52cdce5f48'),
    '2': ObjectId('69d8b3d113314c52cdce5f49'),
    '3': ObjectId('69d8b3d113314c52cdce5f4a'),
    '4': ObjectId('69d8b3d113314c52cdce5f4b')
  }
}
```

2. Aggregation Queries:

(A) Total Sales per Customer:

```
db.orders.aggregate([
{
  $group: {
    _id: "$customer",
    totalSpent: { $sum: "$amount" }
  }
}]
```

```
[
  { _id: 'Amit', totalSpent: 1200 },
  { _id: 'Riya', totalSpent: 2100 },
  { _id: 'John', totalSpent: 300 }
]
```

(B) Total Sales by City:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$city",
      totalRevenue: { $sum: "$amount" }
    }
  }
])
```

```
[
  { _id: 'Pune', totalRevenue: 1200 },
  { _id: 'Mumbai', totalRevenue: 2100 },
  { _id: 'Delhi', totalRevenue: 300 }
]
```

(C) Count Orders by Status:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$status",
      count: { $sum: 1 }
    }
  }
])
```

```
[
  { _id: 'pending', count: 1 },
  { _id: 'cancelled', count: 1 },
  { _id: 'shipped', count: 3 }
]
```

(D) Filter + Group (Only Shipped Orders):

```

db.orders.aggregate([ { $match:
{ status: "shipped" } }, {
  $group: {
    _id: "$customer",
    totalShippedAmount: { $sum: "$amount" }
  }
}
])

```

```

[
  { _id: 'Amit', totalShippedAmount: 1200 },
  { _id: 'Riya', totalShippedAmount: 900 }
]

```

(E) Sort Total Spending (Descending):

```

db.orders.aggregate([
{
  $group: { _id:
"$customer",
    totalSpent: { $sum: "$amount" }
  }
},
{ $sort: { totalSpent: -1 } }
])

```

```

[
  { _id: 'Riya', totalSpent: 2100 },
  { _id: 'Amit', totalSpent: 1200 },
  { _id: 'John', totalSpent: 300 }
]

```

(F) Average Order Value per City:

```

db.orders.aggregate([
{
  $group: {
    _id: "$city",
    avgOrder: { $avg: "$amount" }
  }
}
])

```

```
[
  { _id: 'Pune', avgOrder: 600 },
  { _id: 'Mumbai', avgOrder: 1050 },
  { _id: 'Delhi', avgOrder: 300 }
]
```

3. Indexing Techniques:

(A) Create Single Field Index:

```
db.orders.createIndex({ customer: 1 })
```

```
dlog> db.orders.createIndex({ customer: 1 })
customer_1
```

(B) Create Compound Index:

```
db.orders.createIndex({ city: 1, status: 1 })
```

```
dlog> db.orders.createIndex({ city: 1, status: 1 })
city_1_status_1
```

(C) Index on Date Field:

```
db.orders.createIndex({ date: -1 })
```

```
dlog> db.orders.createIndex({ date: -1 })
date_-1
```

(D) Check Indexes: db.orders.getIndexes()

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { customer: 1 }, name: 'customer_1' },
  { v: 2, key: { city: 1, status: 1 }, name: 'city_1_status_1' },
  { v: 2, key: { date: -1 }, name: 'date_-1' }
]
```

(E) Drop Index:

```
db.orders.dropIndex({ customer: 1 })
```

```
mongo> db.orders.dropIndex({ customer: 1 })
{ nIndexesWas: 4, ok: 1 }
```

4. Using Index + Aggregation Together :

```
db.orders.aggregate([
  { $match: { city: "Pune", status: "shipped" } },
  {
    $group: {
      _id:
"$customer",
      total: {
$sum: "$amount" }
    }
  }
])
```

```
[ { _id: 'Amit', total: 1200 } ]
```

```
db.orders.createIndex({ city: 1, status: 1 })
```

```
mongo> db.orders.createIndex({ city: 1, status: 1 })
city_1_status_1
```